



Configuring and Managing Kubernetes Storage and Scheduling

Implement Kubernetes Persistent Storage Concepts and Management

Why Persistent Storage Is Critical

Hi, and welcome to Configuring and Managing Kubernetes Storage and Scheduling here at Pluralsight. My name is Patrick Rusch, a Senior Consultant and IT instructor from Germany, and I am thrilled to take you on this journey. In this course, we're going to explore two of the most important parts of running real-world Kubernetes applications, persistent storage and intelligence scheduling. You'll learn how to configure storage for your workloads, manage how Kubernetes schedules your pods, and make sure your apps run efficiently and reliably. We'll start with the basics, dive into real-world examples, and see how these concepts apply to everyday development challenges. Throughout the course, you'll see short demos that show exactly how things work inside Kubernetes cluster. Meet Steve, a DevOps engineer at Globomantics. His team has recently containerized several internal applications, databases, analytics tools, and small APIs, all running smoothly in Kubernetes. But after each new deployment, something strange happens. Their applications lose data. Logs disappear, uploaded files vanish, and Steve keeps getting calls from developers wondering where their data went. What's happening here? This is one of the first real-world problems Kubernetes admins encounter. Understanding that by default data inside containers is ephemeral. When a pod goes away, so does its data. When we talk about containers, one of their biggest strengths is how lightweight and disposable they are. You can deploy them, replace them, and move them around the cluster without worrying about the underlying host. But that design also comes with a trade-off. Containers are stateless by default. Every time a pod is restarted, replaced, or rescheduled to a different node, anything stored in the container's file system disappears. For many workloads, that's fine. A simple web frontend or API service can start fresh every time because it doesn't rely on local data. But what about applications that need to keep state? Like database, a file upload service, or even a background job that writes logs. These need data that survives restarts, updates, and even node failures. And that's where persistent storage comes in. Persistent volumes allow Kubernetes to maintain data independently of the pod that use it. So even if the pod is deleted or recreated, the data stays safe and available. In other words, persistent storage brings durability and reliability to workloads that would otherwise lose everything when a container stops. It's one of the key building blocks that turns Kubernetes



from a platform for stateless microservices into one that can also run stateful production-grade applications. Now that we understand why persistent storage matters, let's look at how Kubernetes actually handles it, through persistent volumes, or PVs, and PersistentVolume claims, or PVCs. A PersistentVolume represents a piece of storage that exists in the cluster, provisioned either by an administrator or dynamically through storage classes. You can think of it as a managed disk, NFS share, or any external storage resource that Kubernetes knows how to talk to. It's part of the cluster's infrastructure, available to be claimed and used by workloads when they need it. On the other side, we have PersistentVolume claims. This is what developers or workloads use to request storage. Instead of worrying about where that storage comes from or how it's created, a PVC simply says, I need 5 GB of ReadWriteOnce storage. And Kubernetes then acts as the matchmaker. It finds or provisions a PersistentVolume that fits the request and binds it to the claim. Once that binding happens, the pod can mount the claim just like a regular volume, read and write data, and rely on it to persist even if the pod itself is replaced. And this separation of concerns is really powerful. Cluster admins can manage the underlying storage infrastructure and developers just request the storage they need. The result is flexibility, reusability, and consistency, and that's exactly what we need in a production-grade Kubernetes environment. Let's start with the foundation, the PersistentVolume itself. In this example, we are defining a simple PersistentVolume using a host path which stores data directly on the node's file system. That makes it great for local testing or running Kubernetes on Docker Desktop. The key fields here are the capacity and accessMode. We're giving this volume 1 GB of space and marking it as ReadWriteOnce, meaning it can be mounted by a single node at a time. Notice we also define a storageClassName. This helps Kubernetes identify which storage class this volume belongs to. In this case, we're setting it to local-storage, which could correspond to manually managed disks or directories. In a production environment, you'd likely use a network-backed or dynamically proficient volume. For example, something based on an AWS, EBS, NFS, or another backend. But for learning purposes, hostPath makes it easy to see how Kubernetes ties everything together. And once we apply this manifest, Kubernetes registers the PersistentVolume and marks it as available, waiting for a claim that matches its size, access mode, and storage class. Now that we have a PersistentVolume ready, the next step is to create a PersistentVolumeClaim, the developer's way of requesting storage. Here, the PVC is asking for exactly 1 GB of storage with the same accessMode and storageClassName that your volume provide. Once we apply this claim, Kubernetes looks for an available PersistentVolume that matches the criteria. If it finds one, and it will since we just created it, it automatically binds the claim to that volume. You can verify this binding by running `kubectl get pvc`, which will show the claims status at bound. From this point on, any pod that references this claim will get access to the underlying storage. That means data written by one pod stays available to others, even if the pod is restarted,



rescheduled, or replaced. And this is what makes persistent volumes and claims so powerful. They decouple storage from compute, allowing applications to remain flexible and reliable without losing their data.

Demo: Mounting Persistent Storage in a Pod

Containers are designed to be temporary, which means any data stored inside them disappears when they restart. Many applications, however, need data to persist beyond container lifecycles, and in this demo we'll show you how to provide reliable, persistent storage in Kubernetes. We'll create a PersistentVolume, request it with a PersistentVolumeClaim, and attach it to a pod. Finally, we'll verify that the pod can read and write data, demonstrating how persistent storage ensures your application's data remain safe and accessible. All right, so let's switch over to the terminal and see all this in action. So first we define a PersistentVolume, or PV. This represents a piece of storage in our cluster that can be used by pods. So we use nano pv-demo.yaml. And here we are using a hostPath pointing to /mnt/data on the node. The PV allows read and write access from a single pod at a time. So we're going to save and exit out of here. Next, we create a PersistentVolumeClaim, or PVC. And this is how a pod requests storage from the cluster. So, nano pvc-demo.yaml. Notice that the access mode and storage size matched the PV. Now we save this, so save and exit out of here. Now, we want to create the PV and the PVC in our cluster. So we say kubectl apply -f pv-demo.yaml and kubectl apply -f pvc-demo.yaml. So now we need to check the status to make sure the PVC is bound to the PV. So we hit kubectl get pvc. And we can see that the PVC has the status Bound and it's bound to the volume pv-demo. Next, we'll create a pod that uses this PVC. For simplicity, we're using a stand-alone pod, which makes it easier to verify data persistence. So we say nano pod-demo.yaml. And here the PVC is mounted at /data inside the container, and we're using a simple busybox container that sleeps for an hour. So we save this. Exit out of here. So let's deploy the pod. Kubectl apply -f pod-demo.yaml. Perfect. It says it has the pod created. So let's verify the status of the pod. Kubectl get pods. You can see that we do have a pod-demo pod and it has the status running. Perfect. Now we'll check that the storage actually works. So first we exec into the pod and then we create a test file. To do that, we use kubectl exec as the command, and then we need to specify -it and the pod name, so the pod name is called pod-demo – and then sh because we want to access the shell which is running within the container. From here we can cd into /data. And now we can create a test.txt file. And it should say "Hello, persistent storage!" Save this to a test.txt file. So we're going to verify if we can access the file. So cat test.txt. Perfect. It says "Hello, persistent storage!" So now we can see that the file was created successfully. So we exit out of here, and what we're going to do now is we delete the pod and then redeploy a new pod with the same pod.yaml file. So we say kubectl delete pod and then specify the pod name, pod-demo. And then we redeploy a new pod with the same pod.yaml



file. So, `kubectl apply -f pod-demo.yaml`. It says it had created a new pod. Let's quickly verify that with `kubectl get pods`. And you can see that it is actually a new pod because the age of this pod is only 7 seconds. So once the pod is running again, we can now exec into it and check that the file still exists. So we run `kubectl exec -it`. Provide the pod name. Still `pod-demo`. And then access the shell running within it. And when we now hit `cat /data/test.txt`, we see the same content, confirming that the storage persists across pod deletions. So we successfully created a `PersistentVolume` and `PersistentVolumeClaim`, deployed a pod using the storage, and verified that data persists even after deleting the pod. So you can see persistent storage is essential for applications that need to maintain state, such as databases, log collectors, and any stateful workload in Kubernetes. And that's it for this demo.

StorageClasses and Dynamic Provisioning

As Globomantics continues to scale, Steve's team is onboarding new microservices almost every week. Each of these services needs its own persistent storage for databases, logs, or user uploads, and that quickly turns into a management nightmare when every `PersistentVolume` has to be created manually. It's not just time consuming; it's error-prone and inconsistent. One developer might use a slow disk, another might forget to set the reclaim policy, and before long, storage management becomes a bottleneck. Kubernetes solves this elegantly with storage classes, which act like blueprints for storage. Instead of Steve or his ops team manually provisioning volumes, Kubernetes automatically creates the right kind of storage whenever a developer requests it through a `PersistentVolumeClaim`. And this approach keeps the developer experience simple, just requests what you need while giving the platform team centralized control over performance, cost, and lifecycle policy. Let's take a closer look at how this works. Think of a storage class as a recipe for storage. It defines the how, the provisioner that creates the storage, along with parameters like disk type, replication, or performance class. When a developer creates a `PersistentVolumeClaim` and specifies that storage class, Kubernetes automatically provisions a matching `PersistentVolume`, binds it to the claim, and mounts it to the pod. Behind the scenes it's Kubernetes negotiating with the underlying storage backend, whether that's cloud disk, NFS shares, or local parts, or without the developer ever needing to touch the infrastructure layer. And the result is dynamic provisioning, storage that created only when needed and tailored exactly to the application's requirements. This model brings the same kind of agility to storage that Kubernetes already provides for a compute. And here's an example of a simple storage class definition. We start by giving it a name. In this case, `fast-storage`. The `provisioner` field tells Kubernetes how to create the underlying storage. In production clusters, you'd likely use a container storage interface, or a CSI driver, something like `ebs.csi.aws.com` for AWS or `csi.vsphere.vmware.com` for VMware environments. But for our local Docker



Desktop setup, we'll use the built-in `kubernetes.io/no-provisioning` option which lets us work without any external storage backend. Next, the `volumeBindingMode` controls when volumes are created. The Immediate mode means Kubernetes binds the volume as soon as the claim is made. And finally, the `reclaimPolicy` determines what happens to the underlying volume when the PVC is deleted. Setting it to delete is perfect for dev and test workloads, since it automatically cleans up unused resources, keeping Steve's cluster tidy without manual intervention. For production workloads, though, Steve might switch to retain to prevent accidental data loss. And now that the storage class is defined, developers can use it simply by referencing its name in their PVCs. Here, the PVC called `demo-pvc` requests 1 GB of storage with `ReadWriteOnce` access, meaning it can be mounted by a single node at a time. The key line is `storageClassName: fast-storage`, and this tells Kubernetes which blueprint to use when creating the actual `PersistentVolume`. Once applied, Kubernetes automatically provisions a volume that matches the claim, binds it to the PC, and makes it available to the pod. The developer doesn't need to know anything about the underlying disks or network storage. Kubernetes abstracts all of that complexity away. This not only streamlines developer workflows, but also ensures consistent, policy-driven storage behavior across every microservices in Steve's cluster. In short, storage classes help teams scale storage management with the same automation and flexibility that Kuperittis brings to the containers themselves.

Demo: Create a StorageClass and PVC for Dynamic Provisioning

In this upcoming demo, we'll learn how to create a storage class and a `PersistentVolumeClaim` for dynamic provisioning in Kubernetes. Dynamic provisioning allows Kubernetes to automatically create storage volumes when a PVC is requested. We'll go step-by-step and finish by verifying that the storage works inside a pod. First, we define a storage class. In a production environment, this usually allows dynamic provisioning, but in my Docker Desktop demo here, I'm going to use a static created volume. I prepared that earlier. Nevertheless, it works basically the same. So, `nano` and then `storageclass.yaml`. And the key points in this `.yaml` file are the `provisioner` because the `provisioner` with `no provisioner` tells Kubernetes we won't dynamically create a `PersistentVolume`. Then we do have the `reclaimPolicy`, which is set to `Delete`, means that the PV can be deleted when the PVC is deleted, especially useful for demos. And then we have the `VolumeBindingMode: Immediate`, ensures that the volume is bound as soon as the PVC is created. So we can save and exit out of here. Next step we need to do is we need to apply this storage class to Kubernetes. So, `kubectl apply -f storageclass.yaml`. So we can check with `kubectl get sc` for storage classes, and perfect, we can see that we do have a `demo-storage` storage class, which uses the `provisioner no provisioner`. Now we create a PVC that references the storage



class. So, nano pvc.yaml. And this references our demo-storage storage class. So save and exit out of here. And the next thing we need to do is we need to apply it to our cluster. Perfect. It says it has created a new PersistentVolumeClaim. And with `kubectl get pvc`, we can then check `kubectl get pvc` whether the PersistentVolumeClaim is successfully bound to our PersistentVolume. Perfect. This looks good. So, the PersistentVolumeClaim is bound to a PersistentVolume based on the storage class demo-storage. Next, we deploy a pod that mounts this PVC to actually test our storage. So we specify a .yaml file for our pod deployment, call it pod.yaml. And this specifies a simple pod deployment which is running a demo container using the busybox image and then just sleeps for an hour. Save and exit out of here. Now we need to apply this .yaml file to our cluster, so `kubectl apply -f` and then pod.yaml. It says it has successfully created the pod. And with `kubectl get pods`, you can see that we now have a running demo-pod, which is using the PersistentVolumeClaim we applied earlier. To verify the storage functionality, we can use `kubectl exec -it`, and then the pod name, demo-pod -sh to access the shell, which is running within the container. From here, we can cd into the data directory, and then we can say 'Hello, storage from StorageClass,' for example. We save this into a test.txt file. Verify that we can access it, test.txt. You can see it says Hello, storage from StorageClass. Perfect. Now we can exit out of the container, and what we can now do is we made sure that the file was created successfully, now we're going to go ahead and delete our pod. So, `kubectl delete pod` and then demo-pod because we want to verify if the storage is persistent, right? Once the pod is deleted, we can recreate a new pod based on the same .yaml file. So, `kubectl apply -f jpod.yaml`. With `kubectl get pods`, we can verify that the pod is up and running. Perfect. It says it has created a new pod. It's running. Age is 5 seconds, so we can assume that it is a new pod. And with the same commands, so `kubectl exec -it demo-pod` and then -sh to access the shell. We can now cut into the data directory and take a look at our test.txt file. And again, it says Hello, storage from StorageClass. So we successfully created a storage class, bounded to a PVC or created a PVC relying to this storage class, and it gave us access to storage, persistent storage for our demo-pod, which was running a busybox container.

Implement Advanced Pod Scheduling Techniques

Taints and Tolerations

Welcome to this module on implementing advanced pod scheduling techniques in Kubernetes. In this part of the course, we'll explore how Kubernetes decides where to run your pods and how you, as a cluster administrator or DevOps engineer, can influence that behavior. Scheduling in Kubernetes isn't just about



finding any node with enough CPU and memory. It's about making intentional decisions. Isolating workloads, distributing replicas, and balancing resources used to ensure consistent performance. We'll walk through some of the most important scheduling features, taints, and tolerations, affinity and anti-affinity rules, and resource requests and limits. And by the end, you'll be able to control pod placement with precision. In a perfect world, all your nodes would have identical hardware and identical workloads, but in reality, clusters evolve over time. Nodes differ in capacity, purpose, or even location. At Globomantics, for example, Steve's team runs a mix of compute-heavy analytics workloads and lightweight web services. Without any scheduling rules, Kubernetes tends to place the pods wherever it finds space, which can lead to hot spots, one node overloaded while another sits nearly idle. Smart pod placement solves this problem. By applying scheduling constraints, you help Kubernetes make decisions that align with your workload requirements, improving efficiency, reliability, and overall performance. Let's start with taints and tolerations, one of the most direct ways to influence scheduling. A taint marks a node with a condition that tells Kubernetes only schedule pods here if they can tolerate this taint. Think of it as a selective barrier, for instance, a node reserved for production-only workloads. A toleration is the key that lets a pod bypass that restriction. If the pod's toleration matches the node's taint, it's allowed to run there. And this mechanism gives you control over which workloads can share certain nodes, keeping your environments cleanly separated. And here's an example. We taint a node with the label `dedicated=analytics:NoSchedule`, which tells Kubernetes not to place ordinary pods here. And then afterwards we define a toleration in the pod specification that allows your analytics pod to run on that tainted node. This is especially useful for resource-hungry or sensitive workloads that need isolation. You can dedicate a node to specific purposes, for example, data analytics, security scanning, or GPU processing, while keeping unrelated pods away. And by combining taints and tolerations, you can enforce node-level isolation in your cluster. This approach ensures that your most critical workloads get the resources and stability they need without interference from background jobs or test workloads. And at Globomantics, this helped Steve's operations team eliminate unexpected performance issues. They assigned specific nodes for high performance analytics jobs and left general purpose workloads on the remaining cluster nodes. It's a small configuration change, but it makes a huge impact in a shared production environment.

Demo: Enforcing Node Isolation

In this demo, we learn how to enforce node isolation in Kubernetes using taints and tolerations. This is a common technique for dedicating certain nodes to specific workloads. For example, keeping analytics workloads separate from general application pods. And by the end of this demo, you'll understand how to take the node, create a pod that can tolerate it, and see how the scheduler behaves in



practice. So first, let's check the available nodes in our Kubernetes cluster. Therefore, you can hit `kubectl get nodes`. And in my case, there are 3 nodes. We have the desktop-control-plane, which runs the control plane components, and then I have the desktop-worker and desktop-worker2, and we'll dedicate desktop-worker to analytics workloads only. To do so, we need to apply a taint to this node. Therefore, we hit `kubectl taint nodes`, and then the name of the node you want to taint, in my case, it's desktop-worker. And then we specify the taint, so it's `dedicated=analytics:NoSchedule`. And this taint says do not schedule any pods on this node unless they tolerate this taint. So the key is `dedicated`, the value is `analytics`, and the effect is `NoSchedule`. So that means only pods that explicitly tolerate this taint will be allowed to run on this node, and this is how we start isolating workloads to specific hardware. So we hit Enter, and to verify that the taint is applied, we can run `kubectl describe node desktop-worker`, and then we can `grep` it for Taints. And here we can see that the node is indeed tainted with `dedicated=analytics:NoSchedule`. Great. So the node is now marked for analytics workloads. Next step is to create an analytics pod with a matching toleration. Therefore, we create a .yaml file. Let's call it `analytics-pod.yaml`. And this pod contains two important things. One, a toleration which allows it to run on nodes tainted with `dedicated=analytics`, and a `nodeSelector`, which makes sure it only runs on nodes that are labeled as `dedicated=analytics`. Because otherwise, without a `nodeSelector`, it only tells that they can run on specific nodes, but there's no dedication to it. And this combination ensures the pod is allowed to run on the tainted node, and the scheduler will be scheduling it there specifically. So, save and exit out of here. So before we can apply the pod, we need to label the node so it matches the selector. Therefore, we say `kubectl label node`, and then, again, the node name, it's desktop-worker. And then `dedicated=analytics`. Perfect. So next step now, we can create the pod. So, `kubectl apply -f`, and then our `analytics-pod.yaml`. So now the pod is created, and we can now check where the pod is running. So, `kubectl get pods`. And then with `-o wide`, we get more information. And as you can see, the pod is running and it's scheduled on the desktop-worker. It's not scheduled on the worker2. It's scheduled on the worker because the toleration allowed it and the `nodeSelector` ensured it landed exactly where we want it. And this is the proper way to enforce workload placement when using taints. But what happens without a toleration? So let's go ahead and create another file, which is called `regular-pod.yaml`. So we specify a pod without the `nodeSelector` and without the toleration. Save and exit out of here. And of course, we're going to apply this as well. So, `kubectl apply -f` and on our `regular-pod`. `kubectl get pods -o wide` will exactly tell us where it is. So it will not run on the tainted node, obviously, because it has no toleration, but it can still run on different nodes, right? So in this case, it's the desktop worker2. If all of your nodes are tainted and you're specifying a pod without a toleration, this pod would remain pending since it doesn't have a matching toleration. So in this demo, we saw how taints and toleration work together to isolate workloads in Kubernetes. A taint repels pods from a node, while a toleration lets certain pods bypass that restriction. However, as we saw tolerations only permit scheduling,



they don't force it. To enforce the placement, we combined toleration with a node selector.

Balanced and Efficient Scheduling

Let's look at another pair of scheduling tools, affinity and anti-affinity. While taints and tolerations tell Kubernetes where pods shouldn't run, affinities express preferences for where they should run. A node affinity matches pods to nodes with specific labels, such as hardware type, zone, or role. A pod affinity and anti-affinity control how pods relate to each other. For example, ensuring web app pods run close to backend pods or spreading replicas across nodes for high availability. And together these mechanisms make your deployments both smarter and more predictable. And here's an example of an affinity rule. We are scheduling this pod to a node labeled `kubernetes.io/hostname= worker-1`. You might use this for workloads that require specialized hardware or that need to stay in a specific datacenter zone. And on the flip side, here's a pod anti-affinity example. We use it to ensure that replicas of the same app never land on the same node, and this pattern improves fault tolerance. If one node goes offline, you still have another replica serving traffic. We'll test this in our demo later to see how Kubernetes respects the affinity rules during scheduling. And the last major scheduling control mechanism is resource management. When you define CPU and memory requests, you tell Kubernetes how much capacity your pod needs to run effectively. Limits define the maximum amount a container can use. The scheduler uses these requests to decide where to place the pod, ensuring that the chosen node has enough free resources. And setting these correctly helps prevent resource starvation and ensures fair sharing in a multi-tenant cluster. And in this example, the container requires 250 Mi of CPU and 265 MB of memory and sets the limits as twice that amount. And by defining these values, you are helping the scheduler make informed placement decisions. If you skip them, Kubernetes assumes your pod needs almost no resources, which can lead to too many pods being packed onto a single node. So defining realistic values based on actual metrics ensures that workloads run efficiently and the cluster remains stable.

Demo: Balanced and Efficient Scheduling

In this demo, we'll explore how Kubernetes intelligently schedules workloads to achieve balance and efficiency across the cluster. Specifically, we'll look at three mechanisms that gives us more control over where and how pods run. Node affinity to guide which nodes a pod can be scheduled on, pod affinity to place related pods close to each other, and resource requests and limits to make sure pods get the right amount of CPU and memory. And by combining these features, we can achieve smarter scheduling decisions and ensure that no single



node or workload overwhelms the cluster. Let's start by listing the nodes in our cluster. So with `kubectl get nodes`, you can see that I do have three nodes in my Kubernetes cluster. It is two worker nodes, the `desktop-worker` and the `desktop-worker2`, and 1 control plane. And we'll use the worker nodes to demonstrate how Kubernetes decides where to run specific workloads based on the rules that we define. So the first step is we can use labels for our nodes to then later use affinity rules for the placement. So labels are key-value pairs that describe node characteristics. So here we are now labeling one node for analytics and the other for web application workloads. So we go ahead and say `kubectl label node desktop-worker`, and it becomes the `role=analytics`. And the second one, `kubectl label node desktop-worker2`, we'll get the label `role=web`. So node affinity rules will later use these labels to guide pod placement. So think of it as a setting or setting up the metadata that the scheduler can reason about when making placement decisions. So now, next up is we create a pod with a node affinity and with some resource limits. Therefore, we create a file which is called `analytics-affinity-pod.yaml`. And this pod uses node affinity to ensure it runs only on nodes labeled `role=analytics`. The `requiredDuringSchedulingIgnoredDuringExecution` rule means the scheduler must respect this rule during placement. And the resource requests and limits define how much CPU and memory the pod needs. The request tells Kubernetes the minimum the pod needs to run, and the limit caps how much it can use. These two values help Kubernetes distribute workloads fairly and prevent resource overuse, ensuring balanced scheduling across the cluster. In real environments, these requests contribute to resource quotas, which define how much total CPU and memory each namespace or a team can consume. So, for now, we can save and exit out of here. And then of course we can use `kubectl apply -f` and then the `analytics-affinity-pod.yaml` to actually create the pod. And now the interesting part begins. Now we need to check where the pod is actually running. So `kubectl get pods`, and then `-o wide`. And as you can see, this pod is running on the `desktop-worker` because it matches the label `role=analytics` that we have specified before. And now let me introduce a pod affinity. Therefore, we again create a `.yaml` file. We call it `frontend-sidecar.yaml`. The sidecar pod is configured to run on the same node as any pod labeled `app=frontend`, and the key part is the `topology` key, which says `kubernetes.io/hostname`, which means same node. And this is commonly used when you want related pods like a web app and its local cache or monitoring agent to run together for better performance and lower latency. So, save and exit out of here. Next up, of course, is to apply this. So `apply -f frontend-sidecar.yaml`. See, it has created two pods, and now we need to observe the scheduler's decision, right? So where has the scheduler placed these two pods that we defined in our `frontend-sidecar.yaml`? So, `kubectl get pods -o wide` should tell us the answer. And as you can see, the two pods are running on the same node, the `desktop-worker2`. So, Kubernetes used our defined affinity rules to make those decisions automatically. If a node didn't have enough available CPU memory to satisfy the pod's request, the scheduler would have placed it elsewhere or left it pending, that's how Kubernetes avoids overcommitting



nodes. So what we did is we combined node affinity, pod affinity, and resource limits to achieve a balanced and efficient scheduling, and these features ensures that workloads are placed intelligently, improving performance, stability, and overall cluster efficiency.

Kubernetes Pod Scheduling Mechanisms

How to Work with Kubernetes Scheduling

Welcome back! In this part, we'll take everything you've learned so far and zoom in on one of Kubernetes most powerful features, the scheduling system. Think of scheduling as the traffic controller of your cluster. It decides which node should run each pod, and it plays a huge role in performance, availability, and reliability. We'll start by breaking down how scheduling works, then move into mechanisms like Pod Disruption Budgets and topology spread constraints that keep your applications resilient, even during maintenance or scaling events. At Globomantics, uptime is everything. Steve's team runs customer-facing services that must stay online while they perform system upgrades or patch nodes. But Kubernetes clusters are living systems. Nodes come and go, pods get rescheduled, and workloads move around constantly. And this is where smart scheduling and resilience mechanisms step in. They ensure that workloads aren't only running. They are strategically placed so that your business keeps running smoothly, even when the infrastructure is changing behind the scenes. So the Kubernetes scheduler is the decision-maker that matches pods with nodes. Every time a new pod is created, it goes into a pending state until the scheduler evaluates where it should go. The scheduler examines factors like CPU and memory availability, affinity and anti-affinity rules, taints and tolerations, and even custom priorities. Its goal? To find the most suitable node where the pod can run efficiently, balancing performance, fairness, and reliability across the cluster. Without this logic, pods would end up randomly placed, potentially overloading certain nodes, while others sit idle, and Kubernetes automates this balancing act for you. When the scheduler receives a pod request, it follows two main steps, filtering and scoring. First, it filters out any nodes that don't meet the pod's requirements. For example, maybe a node doesn't have enough memory or it's missing a required label, like `zone=us-east-1`. Then it scores the remaining nodes based on how well they meet their desired conditions. The highest scoring node wins, and the pod is assigned to it. The beauty here is automation. The scheduler continuously optimizes placement as your workloads change, adding new nodes, evicting pods if necessary, and always working toward cluster efficiency. Now, what if the default logic doesn't quite fit your needs? Maybe you want pods running closer to certain data sources, or you'd like to prioritize cost-optimized nodes. Kubernetes allows that. Through the scheduling framework, you can plug in custom logic, filters, scores, or even custom schedulers to influence



pod placement. Some companies build their own schedulers to meet compliance rules or multi-region failover requirements. So while Kubernetes gives you a smart default, it's also flexible enough to adapt to your organization's unique needs, some things Steve at Globomantics really values as their infrastructure keeps expanding. Let's talk about a key feature for maintaining uptime, and this is Pod Disruption Budgets, or PDBs. A PDB defines how many pods in a replicated application can be taken offline at once during voluntary disruptions. Voluntary means actions you or Kubernetes itself trigger, like upgrading a node, draining it, or rolling out a new version of an app. Without a PDB, Kubernetes might accidentally take down all replicas during maintenance. Not ideal for production. And by defining a PDB, you give Kubernetes a safety rule that says always keep at least this many pods running. Imagine Steve's team performing a cluster upgrade at Globomantics. They drain a node to apply updates, but instead of the entire application going offline, Kubernetes follows the PDB's guidance and keeps enough replicas active to maintain service availability. And this ensures rolling upgrades happen safely and predictably. It's a small configuration step, but it makes a huge difference in how your users experience reliability. And here's what that looks like in YAML. We define a budget called the webapp-pdb that targets pods labeled with `app: webapp`. The key setting here is `minAvailable: 2`, which guarantees that 2 pods must always remain available. So, during an upgrade or maintenance window, if taking down another pod would violate that rule, Kubernetes will wait, ensuring your service never drops below the desired availability threshold.

Demo: Protecting Availability with PDBs

In this demo, we'll explore how Kubernetes maintains application availability during voluntary disruptions, such as node maintenance or upgrades. We'll use a Pod Disruption Budget, which, a mechanism that ensures Kubernetes never evicts too many pods at once, protecting our application from downtime. And by the end, you'll understand how to define a PDB and how Kubernetes enforces it. First, we need an application with multiple replicas. In a demo environment, we can use busybox containers because they are tiny, reliable, and don't require an actual web server. Each container will just sleep, which is perfectly enough to simulate a workload. So therefore, we create a .yaml file, which is called web-deployment.yaml. And you can see that we are using a deployment which specifies three replicas of a specific web application. In our case, it's just a simple busybox image. So we're going to save and exit out of here. Next up is, of course, we need to apply this. So, apply `-f webapp-deployment.yaml`. And with `kubectl get pods` I for the label `app=webapp`. So now we do have three pods simulating our web application. Next, we define a PDB to ensure that at least two pods remain available at any time. This protects the application during voluntary disruptions, like node drains or maintenance. Therefore, we create



another .yaml file and we call it webapp-pdb.yaml. So this PDB has the configuration minAvailable: 2. And this tells Kubernetes to never allow more than one pod to be disrupted at any time. So the scheduler and controllers will respect this rule during voluntary actions, such as kubectl drain or rolling updates. You could also use maxUnavailable: 1 to express the same requirement in a slightly different way. It's just two ways to say protect the application. So, save and exit out of here. And, of course, we need to apply this to our cluster, so kubectl apply -f webapp-pdb.yaml. With kubectl get pdb, we can check for the status. And it says Allowed Disruptions 1. It shows how many pods Kubernetes can safely evict at this moment without violating the budget. And since we have 3 replicas and minAvailable2, Kubernetes can evict 1 pod, but will block further evictions to maintain availability. So let's simulate a voluntary disruption by draining a node hosting one of the pods. And this shows you how the PDB is enforced in practice. Before we do that, we hit kubectl get pods once again -o wide. And as you can see, the web app pods are distributed to the desktop-worker and desktop-worker2, so we have 2 nodes running this application. So what we can do now, we can drain one of the nodes with kubectl drain, and then the node name, in this case, desktop-worker --ignore-daemonsets and --delete-emptydir-data. So if we try to drain a node like this that would violate the PDB, we will probably see an error. But let's try it with the desktop worker first. And once it has finished, you can see that the desktop-worker node was drained. If you want to get more details on which pods are protected by your PDB, you can go ahead and say kubectl describe pdb and then our webapp-pdb. And this gives you a clear view of how the pods are handled by this PDB, so how many disruptions are allowed, how many pods are currently up and running, and what is the desired state of your available pods. Let's go ahead and uncordon our desktop worker. And let's try to drain the other worker node in our cluster. So, let me first clear this a little bit. And now we want to try to drain the desktop-worker2. And this is what I was expecting. So we get an error when evicting the pods because it says it cannot evict the pod, as it would violate the pod's disruption budget. So as we have defined in our PDB that we always want to have 2 remaining pods of our deployment, this drain operation will not work because 2 of our pods are running on the desktop-worker2, and the PDB prevents Kubernetes from evicting the pods because it would violate the budget we have created.

Topology Spread Constraints and Demos to Bring It All Together

Another critical concept for reliability is topology spread constraints. Picture this. All of your pods land on the same node or in a single availability zone, and then that zone fails. Suddenly your service disappears. Topology spread constraints solve this by enforcing even distribution of pods across zones, nodes, or other topology domains. And this increases fault tolerance and keeps your workloads balanced, especially in larger clusters. In this YAML, we define a



deployment with six replicas and tell Kubernetes to spread pods evenly across nodes. The `maxSkew: 1` line means that no node should have more than one extra pod compared to others. The `whenUnsatisfiable: DoNotSchedule` policy prevents new pods from being scheduled if that balance can't be maintained, and this setup ensures a fair spread and adds resiliency as a key part of production-grade Kubernetes design. In this demo, we'll explore topology spread constraints, a Kubernetes feature that helps distribute pods evenly across nodes or other failure domains. This ensures that your application remains available even if one node goes down and helps prevent resource hot spots in the cluster. Before creating a deployment, we need to know how many nodes we have and their names. So this is important because the topology spread works by distributing pods across nodes, and by listing the nodes, we can then verify later that pods are spread as expected. So go ahead with `kubectl get nodes`, and we can see a control plane and two worker nodes with the name `desktop-worker` and `desktop-worker2`. So next up is to create a topology spread YAML file. So `nano topology-demo.yaml`, and we want four pods to demonstrate spreading across multiple nodes. So that's why we're specifying four replicas. We also see that it says `topologySpreadConstraints`, and this ensures that the pods are evenly distributed. The `maxSkew: 1` limits the difference in pod count between nodes to 1. The `topologyKey: kubernetes.io/hostname` spreads the pods across nodes. You could also use other labels, for example, zones. And `whenUnsatisfiable: DoNotSchedule`, this specifies that Kubernetes won't violate the constraint. If it can't schedule a pod without breaking the rule, it waits. And the `labelSelector` tells Kubernetes which pods are part of the constraint. Using `busybox` here with a simple `sleep` command keeps this demo lightweight and reliable for the Docker Desktop environment in my case. So, save and exit out of here. So now we can apply this deployment, `kubectl apply -f topology-demo`. And now we can verify the pod distribution. So we go ahead, say `kubectl get pods -o wide` and with the label `app=topology-demo`. And this command shows which nodes the pods are running on. And we can see that Kubernetes tries to respect the topology spread. So the pods are evenly distributed across our nodes. In my case, it has two running pods on different worker nodes and two pending because it will not violate the `maxSkew: 1` when it tries to schedule the pods on other nodes. So, we created our deployment with the spread constraints and Kubernetes distributed the pods across the nodes to maintain high availability, but it also respects the `maxSkew` and `topologyKey`, preventing an uneven distribution and reducing the risk from node failures. In this demo, we'll see how to protect application availability while also spreading pods evenly across nodes. We will combine Pod Disruption Budget with topology spread constraints, and this ensures that Kubernetes keeps a minimum number of pods running and balances them across nodes for high availability. To see how Kubernetes works or combines PDB and topology spreads and respects both configurations, let's go ahead and create a topology spread with `pdb-demo.yaml` file. So, let's call it `topology-pdb-demo.yaml`. And in this case, we are using three replicas to demonstrate distribution across the nodes we have available. Topology spread constraints



ensures that pods are evenly spread. `MaxSkew: 1`, again, prevents more than 1 pod difference between the nodes, and it says `DoNotSchedule` when it's not satisfiable. So it's blocking scheduling that would violate the constraint. So we can save and exit out of here. And then, of course, we can say `kubectl apply -f topology-pdb-demo.yaml`. Next is we want to create a Pod Disruption Budget for our deployment we just created. So, `nano`, and then we call it `topology-pdb-demo`, and then as it is a `pdb` again, `pdb.yaml`. So this PDB ensures that at least two pods are always running, even during voluntary disruptions, like no drains. And combined with the topology spread, this guarantees high availability and balanced placement. So, save and exit out of here, and then, of course, we need to apply it. So, `kubectl apply -f topology-pdb-demo-pdb.yaml`. Perfect. So let's go ahead and verify our settings. So, `kubectl get pods -o wide`. It tells us that we do have two running pods from the `topology-pdb-demo` spread on different worker nodes, and a third one, which is pending because we cannot violate our settings in the topology spread. But what happens if we now simulate a node disruption? Well, let's go ahead and try this out. So we hit `kubectl drain` and then `desktop-worker`, for example. We're going to ignore the daemon sets, and we will delete the `emptydir` data. So let's see what happens because this action will probably violate our PDB, right, because we defined `minAvailable: 2`. But let's see what happens. There it is. It says it cannot evict a pod because it would violate the PDB. So we did see that when combining topology spread constraints and Pod Disruption Budgets, Kubernetes spread the pods evenly across the nodes while protecting availability. And this key pattern for resilient production-ready clusters, ensuring workloads are balanced and high available even in small clusters, like in my demo environment here. And that wraps up our exploration of Kubernetes pod scheduling mechanisms. You've seen how Kubernetes intelligently places workloads, how you can safeguard availability with Pod Disruption Budgets, and how topology constraints help balance your cluster. These tools make your platform predictable, scalable, and resilient, all essential qualities from running modern cloud-native applications. Thanks for learning with me, and keep experimenting in your own clusters to see these mechanisms in action!